LabBook

Título: Trabajo Data Warehouse - Transfermarkt (LabBook)

Autor: Alberto Lillo García y Manuel Caballero Bonilla

Asignatura: Bases de Datos Avanzadas.

Fecha: Marzo de 2026.

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

esi Escuela Superior de Informática

ÍNDICE

1. NOTA 5

2. 6 DE FEBRERO HASTA EL 8 DE FEBRERO 5

3. 9 DE FEBRERO HASTA EL 12 DE FEBRERO 6

3.1. Tablas de Dimensiones (Contexto) 6

3.2. Tablas de Hechos (Métricas) 6

4. 13 DE FEBRERO HASTA EL 19 DE FEBRERO 8

5. 20 DE FEBRERO HASTA EL 25 DE FEBRERO 9

5.1 Contexto y Planificación 9

5.2 Arquitectura Modular del Sistema ETL 9

5.3 Estrategia de Tratamiento de NULLS 10

5.4 Proceso de carga y Resultados Cuantitativos 11

5.5 Sistema de Validación y Aseguramiento de Calidad. 12

5.6 Problemas Encontrados y Soluciones 12

6. 27 DE FEBRERO AL 28 DE FEBRERO 13

6.1 Contexto y Planificación 13

6.2 Estructura del Corpus de Consultas 14

6.2.1 Secciones 1-7: Operaciones OLAP Puras (23 consultas) 14

6.2.2 Sección 8: Análisis Estratégicos Combinados (5 consultas) 14

6.2.3 Sección 9: Consultas Estrella (10 consultas) 14

6.2.4 Sección 10 – 12: Extensiones (10 consultas) 15

6.3 Técnicas SQL Avanzadas Empleadas 15

6.4 Problemas Encontrados y Soluciones 16

7. 1 DE MARZO AL 2 DE MARZO 16

7.1 Contexto y Planificación 17

7.2 Estructura del Notebook de Visualización 17

7.3 Decisiones de Diseño Visual 18

7.4 Problemas Encontrados y Soluciones 18

8. BIBLIOGRAFÍA 19

1. NOTA

Todos los grandes detalles se pueden encontrar en la documentación del trabajo y repositorio de nuestro proyecto.

2. 6 DE FEBRERO HASTA EL 8 DE FEBRERO

En estos tres primeros días nos centramos en la fase inicial del proyecto: seleccionar el *dataset*, conocerlo a fondo y preparar el entorno técnico necesario antes de empezar a modelar o programar. Esto nos llevó aproximadamente unas **8 horas repartidas en 3 días**.

Lo primero que hicimos fue decidir y justificar el *dataset* que íbamos a utilizar. Tras revisar varias opciones de *OpenData* (estadísticas deportivas, clima, economía, etc.), elegimos "*Footbal Data from Transfermarkt*" porque cumple perfectamente con los requisitos del enunciado del trabajo, además de con lo que queremos demostrar y nos apasiona.

Es un *dataset* público, actualizado, con licencia CC0 (dominio público), de gran volumen (más de 5,7 millones de registros) y, sobre todo, muy rico en relaciones y granularidades diferentes: partidos completos, apariciones de jugadores, eventos minuto a minuto, valoraciones de mercado y transferencias. Esto nos permite aplicar de forma realista conceptos clave del curso como ETL, diseño multidimensional (estrella/constelación, copo de nieve), *OLAP* y *data mining*, y generar preguntas analíticas útiles (tendencias de valoraciones, impacto de fichajes, rendimiento por posición y liga, etc.). Además, su estructura relacional bien definida y su enfoque en ligas europeas lo hacen ideal para un proyecto educativo que quiere ser práctico y visual.

Una vez elegido el *dataset*, realizamos un estudio profundo de los 10 archivos *.csv*: revisamos su estructura, tamaño total (556 MB), número de registros, calidad de los datos (pocos nulos, buena consistencia en ID's y fechas) y las relaciones entre tablas mediante claves foráneas. Esta exploración inicial fue fundamental porque nos permitió entender desde el principio los diferentes niveles de granularidad y planificar mejor el modelo dimensional posterior.

A continuación, configuramos el entorno de desarrollo:

- Creamos el repositorio en *Github* para tener control de versiones y poder trabajar los dos de forma ordenada desde el primer día.
- Instalamos Python 3.12 y librerías principales comentadas en documento de planificación (*Pandas*, *SQLALchemy*)...
- Instalamos *PostgreSQL* como base de datos principal. Pero no realizamos la creación y configuración de la base de datos del proyecto y probar conexión con *SQLAlchemy* hasta tener un modelado definido.

Decisiones tomadas y su justificación:

- Elegimos *Transfermarkt* porque combina volumen, calidad, relaciones y temática interesante, permitiéndonos demostrar todos los bloques del curso de forma coherente.
- Optamos por *PostgreSQL* como motor principal por su buen soporte a consultas analíticas y datos temporales, pero dejamos preparadas *MySQL* y *SQLite* como alternativas por si surge cualquier problema.
- Acordamos usar *Microsoft Word* para el *Lab-Book* y el informe final, reservando *Jupyter Notebook* para más adelante una vez nos introduzcamos en detalle y consultas.

Problemas encontrados y soluciones:

No encontramos incompatibilidades importantes. Todo se instaló y funcionó correctamente. Como medida preventiva, documentamos las alternativas (*SQLite* y *MySQL*) por si en algún momento necesitamos cambiar de motor.

3. 9 DE FEBRERO HASTA EL 12 DE FEBRERO

Se investigó más en detalle acerca del modelado dimensional, tras dicha investigación para adquirir mayores conocimientos de modelado se realizó el "**Diseño del Esquema (Modelado)**" de nuestro *Data Warehouse* lo que nos llevó en total a una *dedicación aproximada* de unas **8 horas**.

Tras examinar exhaustivamente nuestros datos para realizar nuestro modelado, hay un detalle importante que debíamos tener en cuenta: como tenemos datos **con diferentes niveles de granularidad** (partidos, eventos minuto a minuto, fichajes anuales y valoraciones de mercado en el tiempo), planteamos en un primer momento **un simple diseño en estrella**, pero vimos que no era suficiente y vimos que el diseño óptimo no es una sola estrella, sino un **Esquema en Constelación con dimensiones tipo Estrella** (Diseño en estrella/Esquema en estrella con más de una tabla de hechos). Esto significa que, dado el conjunto de datos complejo, tendremos la combinación de múltiples esquemas en estrella, donde varias **tablas de hechos** (*centro de la estrella*) compartirán las mismas **tablas de dimensiones** (*puntas de la estrella*).

3.1. Tablas de Dimensiones (Contexto)

Las tablas de dimensiones son el "*cómo*" y el "*por qué*" analizamos los datos. En nuestro caso, como tenemos varios procesos de negocio con granularidades distintas (partidos, apariciones, eventos, transferencias y valoraciones), decidimos crear dimensiones compartidas que sirvan a todas las tablas de hechos. Esto es precisamente lo que permite el esquema en constelación: las dimensiones son reutilizables y se mantienen estables, mientras cada hecho mide un aspecto diferente del fútbol. Elegimos estas cinco dimensiones porque cubren todos los puntos de vista necesarios del *dataset* y evitan repetir información.

- **Dimensión Players** (Basada en *players.csv*).
- **Dimensión Clubs** (Basada en *clubs.csv*).
- **Dimensión Competitions** (Basada en *competitions.csv*).
- **Dimensión Games** (Basada en *games.csv*).
- **Dimensión Date** (Tabla generada).

3.2. Tablas de Hechos (Métricas)

Las tablas de hechos contienen las medidas numéricas que realmente queremos analizar (los "*qué*" del negocio). Dado que el *dataset* tiene procesos con granularidades muy diferentes (un partido completo, una aparición de jugador, un evento minuto a minuto, un fichaje o una valoración de mercado), crear una única tabla de hechos habría generado una tabla gigantesca y poco usable. Por eso optamos por cinco tablas de hechos separadas que comparten las mismas dimensiones. Cada una se centra en un proceso concreto del fútbol, permitiendo consultas más rápidas y naturales. Todo ello basándonos en los datos de los *csv* correspondientes. A continuación, se presenta una tabla con todos los detalles acerca de las tablas de hechos creadas:

Nombre Tabla de Hechos	Intención	Formación	PK	Granularidad	FK's	Atributos Degenerados	Medidas
Fact Games	Resumen de lo ocurrido en cada partido.	Unión de *games*.csv y *club_games*.csv con desnormalización.	ID del partido (PK y FK a Dim Games)	Un registro por partido.	*game_id, competition_id, home_club_id, away_club_id, date_id.*	-	*home_club_goals, away_club_goals, is_win (Para el equipo Local)*
Fact Appearances	Registra el rendimiento del jugador en cada partido.	Combinación de *appearances*.csv y *game_lineups*.csv con desnormalización.	ID de *apperarance*	Un registro por jugador en un partido.	*game_id, player_id, club_id, competition_id, date_id.*	*type* (titular o suplente), *position* (posición específica jugada ese día), *team_captain* …	*minutes_played, goals, assists, yellow_cards, red_cards*
Fact Game Events	Registra el rendimiento del jugador en cada partido.	Basada en *game_events*.csv	ID de ese suceso	Un registro por evento dentro de un partido.	*game_id*, *club_id*, *player_id*, *player_in_id* (si es sustitución), *player_assist_id* (si es gol), *date_id.*	*type* (tipo de evento), *description*.	*minute*
Fact Transfers	Representa los traspasos de jugadores.	Basada en *transfers*.csv.	*transfer_id* (atributo auto incremental generado)	Un registro por traspaso.	*player_id*, *from_club_id*, *to_club_id*, *transfer_date_id*.	*transfer_season*.	*transfer_fee*, *market_value_in_eur*
Fact Player Valuations	Histórico del valor de mercado de un jugador.	Basada en *player_valuations*.csv.	*valuation_id* (atributo auto incremental generado)	Un registro por jugador, club y fecha.	*player_id*, *club_id* (como *current_club_id*), *competition_id*, *date_id*.	-	*market_value_in_eur*

Figura 1: **Tabla con los Detalles de Nuestras Tablas de Hechos.**

Tras diseñar el esquema en estrella nos planteamos el **esquema en copo de nieve**, pero nos dimos cuenta enseguida que no sería lo mejor para nuestro caso debido al elevado número de registros, porque a la hora de hacer **consultas** y **desnormalizar todavía más las dimensiones** para luego mostrar o visualizar algo tendríamos un impacto negativo en rendimiento/velocidad. Explicando esto más en detalle, si usáramos copo de nieve, tendríamos que fragmentar tablas como la de dimensión de los jugadores; por ejemplo, sacando el país de nacimiento a una nueva hipotética tabla de países, o en el caso de la tabla de dimensiones de clubes, sacando el estadio a una nueva hipotética tabla de recintos... Esto obligaría al motor de base de datos que utilicemos a realizar múltiples saltos encadenados (más consultas *SQL* de las necesarias) cada vez que queramos calcular algo tan habitual como **cuántos goles marcó un jugador de cierta nacionalidad en un estadio concreto**, lo cual, es evidente que "*castigaría*" y ralentizaría enormemente el sistema.

Además, del rendimiento técnico, el esquema en estrella nos ofrece una ventaja decisiva en cuanto a la **usabilidad** y la **agilidad** del análisis. Cuando usemos nuestro *Data Warehouse* junto a herramientas de *Business Intelligence* como *Power BI* o *Plotly* para crear *insight's* y así responder preguntas de negocio, querremos que la exploración de los datos sea **intuitiva**. Por ejemplo, al tener una tabla de dimensión de jugador ancha y completa, podremos simplemente buscar la columna de: la posición, la pierna buena o el

país de ciudadanía y usar directamente dichos datos sin tener que descifrar y navegar por un laberinto de subtablas relacionadas. Esta simplicidad estructural creemos que nos **reduce drásticamente** el tiempo de desarrollo de los reportes que generemos (utilizaciones) y la probabilidad de cometer errores cruzando datos, permitiéndonos, centrarnos en lo importante: **descubrir patrones en el juego** o **analizar el mercado de fichajes**.

Por último, para concluir con la justificación de nuestro modelado, es importante el haber analizado este diseño desde la realidad tecnológica actual y desmitificar la razón por la que **históricamente** se usaba el copo de nieve: **el ahorro de espacio de almacenamiento**. Es cierto que en nuestra dimensión de jugadores la palabra "Spain" o "Centre-Forward" se repite cientos de veces, algo que el copo de nieve **evita aislando esos términos**. Sin embargo, hoy en día el almacenamiento es **extremadamente barato en comparación a antaño** y las bases de datos analíticas utilizan tecnologías de compresión columnar **que manejan esa redundancia de texto de forma automática y muy eficiente**. En otras palabras, el insignificante espacio en disco que ahorraríamos normalizando los países, las competiciones o los agentes de los jugadores… no justifica en absoluto el impacto negativo que sufriríamos en lentitud y en la complejidad de las consultas que realicemos.

Modelado *DW Transfermarkt*

4. 13 DE FEBRERO HASTA EL 19 DE FEBRERO

Durante el período comprendido entre el 13 y el 19 de febrero, en el cual se invirtió una dedicación aproximada de **7 horas** de trabajo, el desarrollo del proyecto experimentó una ligera desviación respecto a la planificación temporal original. Dicha alteración se derivó de factores externos vinculados al entorno universitario, los cuales imposibilitaron la ejecución íntegra y completa tal como queríamos del proceso **ETL** (***E**xtract*, ***T**ransform*, ***L**oad*) bajo condiciones óptimas. No obstante, el avance logrado durante este lapso fue sustancial, lo que permitió mitigar el impacto de este retraso mediante la consolidación de la infraestructura metodológica y técnica. De este modo, se garantizó que los esfuerzos posteriores a la fecha señalada se orientaran de manera exclusiva a la ejecución, ajuste y optimización del sistema para la obtención final de los resultados que estamos buscando, eludiendo la necesidad de reanudar el desarrollo desde sus fases preliminares.

En primer término, se llevó a cabo la provisión y configuración de la base de datos en *PostgreSQL*, entorno destinado a albergar el *Data Warehouse*. Esta infraestructura quedó debidamente acondicionada para soportar cargas masivas de información y consultas analíticas complejas. Sobre esta base de datos se ejecutó el script de definición de datos denominado ***ddl_dwh_schema.sql***, el cual materializa el diseño dimensional previamente establecido. Dicho esquema se ha sustentado en dimensiones conformadas; tales como jugadores (*players*), clubes (*clubs*), competiciones (*competitions*), partidos (*games*) y tiempo (*date*), así como en tablas de hechos independientes correspondientes a cada dominio de análisis (partidos, apariciones, eventos, transferencias y valoraciones). Adicionalmente, el diseño físico incorpora claves primarias, claves foráneas e índices *estratégicamente* definidos para optimizar las operaciones de reunión (*joins*) características de los sistemas ***OLAP*** (***O**nline **A**nalytical **P**rocessing*), dotando al modelo de la coherencia y robustez estructural exigidas para el análisis una vez culminada la ingesta de datos.

De manera paralela, se progresó en el diseño y codificación del pipeline ***ETL*** utilizando el lenguaje *Python*, estableciendo una arquitectura modular organizada en torno a las entidades del modelo (dimensiones y hechos) y provista de una **jerarquía de ejecución estructurada**. Esta base de código **no** representa un estadio de desarrollo **incipiente**, sino un hito **consolidado** que faculta la carga independiente de cada **entidad**. En virtud de este orden, las fases subsiguientes se focalizarán en maximizar el desempeño; concretamente, en la **optimización de las cargas por lotes** (*batch processing*), los **tiempos de respuesta**, el **robustecimiento de los protocolos de validación** y el **refinamiento de las reglas de transformación** para asegurar la consistencia y reproducibilidad de los resultados.

5. 20 DE FEBRERO HASTA EL 25 DE FEBRERO

Durante el período comprendido entre el 20 y el 25 de febrero se llevó a cabo la implementación completa del proceso ***ETL*** (Extract, Transform, Load) para el ***Data Warehouse*** de fútbol. Esta fase requirió **14 horas de trabajo efectivo**, distribuidas en sesiones de desarrollo, pruebas de validación y resolución de incidencias. Hubo una pequeña desviación respecto a la planificación (13 de febrero a 19 de febrero) explicada en el apartado anterior. Esta fase representa la materialización del diseño dimensional establecido previamente, transformando la infraestructura conceptual en un sistema operativo capaz de almacenar y estructurar ***3.176.424 registros*** del *dataset* de Transfermarkt bajo el esquema en constelación diseñado.

5.1 Contexto y Planificación

El proceso ***ETL*** constituyó el núcleo operativo de nuestro proyecto: sin él, el modelo dimensional diseñado en la anteriormente era mera y únicamente una estructura conceptual; su función: **extraer** los datos crudos del *dataset* de *Transfermarkt*, aplicar todas las **transformaciones** necesarias (limpieza, normalización, resolución de nulos y construcción de claves sustitutas) y **cargarlos** para tener el correcto esquema *dwh* de *PostgreSQL* respetando las restricciones de integridad referencial de nuestro modelado.

Hay que comentar que la arquitectura ***ETL*** se ha diseñado siguiendo una aproximación modular, donde cada entidad del modelo dimensional (5 dimensiones y 5 hechos) dispone de su propio *script* en *Python* independiente. Esta decisión arquitectónica persigue dos objetivos fundamentales: facilitar el mantenimiento individual de cada proceso y permitir la ejecución selectiva de cargas cuando fuese necesario.

5.2 Arquitectura Modular del Sistema ETL

El sistema ***ETL*** se compone **principalmente** de 11 *scripts Python*, organizados jerárquicamente según las dependencias de claves foráneas entre tablas. Adicionalmente, dos módulos transversales (***config.py*** y ***null_handler.py***) centralizan la configuración y las reglas de negocio aplicables a todas las entidades.

Nombre Script	Descripción
generate_dim_date.py	Generación programática de la dimensión temporal desde el año 2000 hasta 2030, sin dependencias de archivos .CSV, incluyendo atributos derivados como trimestre, día de la semana e indicador de fin de semana.
etl_dim_competitions.py	Carga de competiciones desde competitions.csv.
etl_dim_clubs.py	Carga de clubes desde clubs.csv, más inserción del registro centinela club_id = -1
etl_dim_players.py	Carga de jugadores desde players.csv, aplicando cálculo de edad a partir de la fecha de nacimiento, más registro centinela player_id = -1.
etl_dim_games.py	Carga de partidos desde games.csv, actuando como dimensión degenerada con atributos descriptivos propios.

Figura 2: *Tabla Descriptiva de los Scripts ETL para la Generación de las Tablas de Dimensiones.*

Nombre Script	Descripción
etl_fact_games.py	Desnormalización de *games.csv* y *club_games.csv*, generando métricas agregadas.
etl_fact_appearances.py	Fusión de *appearances.csv* y *game_lineups.csv* mediante LEFT JOIN.
etl_fact_game_events.py	Carga de eventos minuto a minuto desde *game_events.csv* (goles, tarjetas, sustituciones).
etl_fact_transfers.py	Transformación de *transfers.csv* con *parsing* de valores monetarios en formato texto ("€50.00m" → 50000000) y normalización de temporadas.
etl_fact_player_valuations.py	Carga de valoraciones históricas desde *player_valuations.csv* con interpolación temporal.

Figura 3: *Tabla Descriptiva de los Scripts ETL para la Generación de las Tablas de Hechos.*

Nombre Script	Descripción
run_etl_full.py	Coordinador maestro que ejecuta los 10 scripts ETL en orden de dependencias, implementando truncado previo de tablas (*truncate_all_tables*()), *logging* completo y gestión centralizada de errores.

Figura 4: *Tabla Descriptiva del Script cuya Función es la de ser el Orquestador Principal.*

Nombre Script	Descripción
config.py	Centraliza la conexión *PostgreSQL* mediante *SQLAlchemy* (*get_engine*()), define rutas absolutas a los 10 archivos .CSV y establece *BATCH_SIZE*=10000 para la optimización de inserciones por lotes.
null_handler.py	Implementa la estrategia de *Kimball*[1][2][3] de tratamiento de valores ausentes mediante seis constantes tipadas y dos funciones de aplicación/validación.

Figura 5: *Tabla Descriptiva de los Scripts con los Módulos de Soporte.*

5.3 Estrategia de Tratamiento de NULLS

Los *.CSVs* de origen contenían *NULLs* masivos: ***coach_name*** (439), ***agent_name*** (16.019), ***contract_expiration_date*** (12.091), ***player_assist_id*** (681.472), ***from_club_id*** (51.226), entre otros. Siguiendo la metodología de Kimball[1][2][3], se diseñó el módulo ***null_handler.py*** como *Single Source of Truth*[1] para la normalización de valores nulos.

Taxonomía de 8 categorías con sus valores por defecto:

Categoría	Valor por defecto	Aplicación
Texto en dimensiones	'N/A'	Atributo nunca registrado en la fuente.
Texto en hechos	'Unknown'	Detalle del evento no capturado.
Medida agregable	0	*goals, assists, minutes_played*, etc; ausencia = no ocurrió
Atributo clasificador	-1	*minute, attendance, squad_size*; valor imposible en el dominio
Valor monetario	-1	*transfer_fee*, *market_value_in_eur;* distingue "gratis" (0) de "sin dato" (-1)
FK opcional	-1	*player_assist_id, from_club_id*; apunta al registro centinela
Booleano	False	*is_major_national_league, team_captain*
Fecha desconocida	'1900-01-01'	*date_of_birth* sin dato.
Fecha sin vencimiento	'9999-12-31'	*contract_expiration_date* indefinido.

Figura 6: *Tabla Descriptiva de la Asignación de cada Valor para cada Tipo de Dato NULL.*

El diccionario *NULL_RULES* define las reglas para cada una de las 10 tablas, clasificando cada columna en su categoría correspondiente. La función ***apply_null_rules(df, table_name, is_dimension)*** aplica *fillna()* por categoría en una sola llamada, y ***validate_no_nulls(df, table_name)*** verifica que no quede ningún *NULL* residual.

Esta estrategia se complementa con **registros centinela** insertados manualmente en las dimensiones candidatas principales:

- ***Dim_clubs:*** *club_id = 1, club_name = 'Desconocido'*
- ***Dim_players:*** *player_id = -1, player_name = 'Desconocido'*

[1] *Single Source of Truth: concepto de gestión de datos donde toda la información crítica de un ámbito se consolida en un solo repositorio centralizado y confiable.*

Estos registros permiten que todas las claves foráneas opcionales (por ejemplo, *from_club_id* en transferencias de jugadores procedentes de clubes no europeos) apunten a un registro válido, eliminando la necesidad de *LEFT JOINs* y garantizando que todas las consultas ***OLAP*** utilicen exclusivamente *INNER JOINs*, mejorando así el rendimiento y la predictibilidad de los resultados.

Las funciones implementadas son:

- ***apply_null_rules(df, table_name, is_dimension):*** Aplica sustituciones diferenciadas según tipología de tabla.
- ***validate_no_nulls(df, table_name):*** Lanza excepción si detecta *NULLs* residuales pre-carga.

Esta estrategia fue documentada exhaustivamente en el archivo técnico ***ESTRATEGIA_NULLS_DWH.md.***

5.4 Proceso de carga y Resultados Cuantitativos

El script orquestador ejecuta el pipeline en 4 fases secuenciales mediante *subprocess.run()*:

- **Fase 0 (Validación previa)**: Ejecuta ***config.py*** para verificar la existencia de los 10 *CSV* y la conexión a PostgreSQL.
- **Fase 1 (Truncado y dimensión fecha)**: *truncate_all_tables()* ejecuta ***TRUNCATE TABLE dwh.{tabla} CASCADE*** sobre las 10 tablas (hechos primero, luego dimensiones); después ejecuta ***generate_dim_date.py.***
- **Fase 2 (Dimensiones)**: Ejecuta secuencialmente ***etl_dim_clubs.py, etl_dim_competitions.py, etl_dim_players.py*** y ***etl_dim_games.py.***
- **Fase 3 (Hechos)**: Ejecuta secuencialmente los 5 scripts de hechos.
- **Fase 4 (Validación)**: Ejecuta ***validate_dwh.py.***

El tiempo total de ejecución puede variar de **3 minutos a 15 minutos**, siendo considerado, en un sistema con PostgreSQL 18 local, un tiempo óptimo dado el volumen de datos procesado.

Tabla	Registros Cargados	Fuente Principal
dim_date	11.323	Generación programática
dim_competitions	44	competitions.csv
dim_clubs	440	clubs.csv + centinela
dim_players	32.602	players.csv + centinela
dim_games	58.495	games.csv
fact_games	58.495	games.csv + club_games.csv
fact_appearances	1.628.836	appearances.csv + game_lineups.csv
fact_game_events	809.989	game_events.csv
fact_transfers	79.594	transfers.csv
fact_player_valuations	496.606	player_valuations.csv

***Figura** 7: Tabla con los Registros Finales Cargados en las Dimensiones y Hechos.*

La causa principal de filtración fue ***dim_games***: de los 74.026 partidos del *CSV* original, 15.531 fueron descartados porque sus *home_club_id* o *away_club_id* no existían en ***clubs.csv*** (equipos de ligas menores no incluidos en el *dataset*). En cascada, los hechos que referenciaban esos ***game_id*** también fueron filtrados.

Para las FK opcionales, se adoptó un enfoque diferente: en lugar de eliminar registros, se redirigieron al centinela (-1).

El resultado fue de una carga de **3.176.424** registros frente a los más de **5,7 millones de registros** originales, lo que evidenció la importancia de: transformar, analizar y tratar los datos provenientes de la web.

5.5 Sistema de Validación y Aseguramiento de Calidad.

Para garantizar la integridad estructural y semántica del DW se desarrollaron cuatro scripts de validación complementarios que ejecutan varias pruebas automatizadas:

Validate_dwh.py:

Ejecuta consultas SQL organizadas en cinco categorías:

- Conteo de registros por tabla
- Integridad referencial mediante *LEFT JOIN* buscando huérfanos en *fact_games* → *dim_competitions, dim_clubs y dim_date.*
- *NULLs* en *PKs* y *FKs* críticas.
- Rangos de fechas.
- Estadísticas básicas (promedio goles/partido, victorias locales/empate/visitante)

Validate_full_etl.py:

- Validación exhaustiva que compara conteos de los *CSV* originales vs *PostgreSQL* para cada tabla (con porcentaje de tasa de carga), verifica las 5 medidas calculadas de ***fact_games*** (comprueba que *goal_difference = home_club_goals – away_clubs_goals* para cada registro), valida la unicidad de las *PKs* artificiales (*transfer_id, valuation_id*), comprueba la integridad referencial de 6 relaciones *FK* y verifica la desnormalización (columnas de ***club_games.csv*** presentes en ***fact_games***, columnas de ***game_lineups.csv*** presentes en ***fact_appearances***).

Check_nulls_final.py:

Recorre las 10 tablas consultando *information_schema.columns,* ejecuta *COUNT (*) WHERE col IS NULL* para cada columna y confirma el resultado final: **0 NULLs** absolutos en todas las columnas de todas las tablas del DW.

Reset_dwh.py:

Utilidad de mantenimiento que ejecuta *TRUNCATE CASCADE* respetando dependencias. Incluye confirmación interactiva (*input ('Escribe BORRAR para confirmar')*) para prevenir borrados accidentales. Aunque este script fue menos utilizado al implementar el *TRUNCATE* en el script de ejecución global.

5.6 Problemas Encontrados y Soluciones

Problema 1: Violaciones de Integridad Referencial en Transferencias.

- **Descripción**: Durante las primeras ejecuciones, algunos registros en ***fact_transfers*** referenciaban clubes inexistentes en ***dim_clubs.***
- **Causa**: Datos inconsistentes en los *CSV* originales donde existían transferencias a clubes no documentados en ***clubs.csv.***
- **Solución**: Implementación de lógica en ***etl_fact_transfers.py*** que asigna *club_id = -1* (registro centinela) cuando la clave foránea no encuentra correspondencia en la dimensión.

Problema 2: Rendimiento en Carga de Hechos.

- **Descripción**: La tabla ***fact_appearances*** con casi 2 millones de registros tardaba más de 15 minutos en cargarse.
- **Solución**: Implementación de carga por lotes mediante *pandas.to_sql()* con parámetro *chunksize=10000*, reduciendo el tiempo a 2 minutos.

Problema 3: Desbordamiento de pd.Timestamp para la Fecha Centinela '9999-12-31'.

- **Descripción**: Al procesar la columna *contract_expiration_date* con *pd.to_datetime()*, pandas lanzaba un error de desbordamiento para el valor centinela *'9999-12-31'.*
- **Causa**: pandas representa fechas internamente con resolución nanosegundo, cuyo límite máximo es aproximadamente el año 2262. La fecha 9999-12-31 supera ese límite.

- **Solución**: La columna se mantuvo como tipo *object* y el valor se asignó directamente como *string '9999-12-31',* que PostgreSQL acepta nativamente como tipo *DATE*.

Problema 4: Valores de transfer_fee en Formato String No Numérico.

- **Descripción**: La columna *transfer_fee* en ***transfers.csv*** contenía cadenas como *"€50.00m"*, *"€500k"* o *"free"* en lugar de valores numéricos en EUR.
- **Causa**: *Transfermarkt* almacena los importes en el formato legible de su web, sin normalizar a valor numérico.
- **Solución**: Implementación de la función *parse_transfer_fee()* en ***etl_fact_transfers.py***, que aplica la expresión regular ***r'([\d.]+)\s*([mk])?'*** para extraer valor y unidad (millones m/ miles k), convierte casos especiales ('free', 'loan') a 0, y devuelve el importe en EUR.

Problema 5: Formato de Temporada "23/24" Incompatible con Operaciones Numéricas.

- **Descripción**: La columna *transfer_season* en ***transfers.csv*** contenía cadenas en formato *"23/24"*, impidiendo comparaciones y agrupaciones numéricas en las consultas ***OLAP***.
- **Causa**: Convención de nomenclatura de Transfermarkt que abrevia ambos años de una temporada.
- **Solución**: Implementación de la función *parse_season()* en ***etl_fact_transfers.py***, que divide el string por *'/'* y convierte el año corto al año completo de inicio de temporada (*"23/24"* → *2023*), manteniendo consistencia con el campo *season* de ***dim_games***.

6. 27 DE FEBRERO AL 28 DE FEBRERO

Durante los días 27 y 28 de febrero, con una dedicación aproximada de **8 horas repartidas en dos sesiones** se llevó a cabo la fase de explotación analítica del ***Data Warehouse*** mediante el diseño, implementación y documentación de un corpus completo de consultas ***OLAP*** sobre PostgreSQL. Como ya llevábamos un desfase en la planificación desde la anterior fase, esta también lo sufrió, ya que estaba planificada para el 20 al 23 de febrero, se retrasó 4 días. Sin embargo, conseguimos el objetivo de la etapa en 2 días a diferencia de la planificación inicial (4 días). Esta fase constituye la materialización del objetivo central del proyecto: demostrar la utilidad analítica del esquema en costelación construido, respondiendo a preguntas de negocio concretas a partir de los ***3.176.424 registros*** cargados.

6.1 Contexto y Planificación

Habiendo finalizado la fase *ETL* con el *DW* completamente operativo y validado, el objetivo de esta etapa consistió en diseñar un conjunto exhaustivo de consultas que cubriera todas las operaciones ***OLAP*** fundamentales estudiadas en el curso, garantizando al mismo tiempo que cada consulta respondiera a una pregunta analítica real del dominio del fútbol europeo y fuera directamente presentable.

Antes de empezar a escribir código *SQL* se tomó la decisión de organizar el trabajo en bloques temáticos diferenciados, para no mezclar consultas de carácter didáctico con consultas orientadas a resultado. Esto dio lugar a la estructura final del fichero ***consultas_olap.sql***.

6.2 Estructura del Corpus de Consultas

El fichero se organiza en **12 secciones** agrupadas en **4 bloques funcionales**:

Bloque	Secciones	Consultas	Descripción
OLAP puras	1 - 7	23	Una operación por sección, máximo didactismo.
Análisis combinados	8	5	Varias operaciones integradas por caso de uso.
Consultas estrella	9	10	*Insights* directamente presentables.
Extensiones	10 – 12	11	Auditoría técnica, jugadores individuales y equipos.
Total		49	

***Figura 8**: Tabla con el Recuento de Consultas OLAP Realizadas.*

6.2.1 Secciones 1-7: Operaciones OLAP Puras (23 consultas)

- ***SLICE (3):*** fija una única dimensión (temporada 2023, Premier League (GB1) y FC Barcelona (*club_id = 131*), y deja el resto del cubo libre.
- ***DICE (3):*** cruza dos o más dimensiones simultáneamente (goles de españoles en las Cinco Grandes Ligas entre 2020 y 2024, transferencias Bundesliga → Premier superiores a 10 Millones €, tarjetas rojas en fin de semanas).
- ***ROLL-UP (3):*** agrega ascendentemente con *ROLLUP()* por jerarquía temporal (mes → trimestre → año), geográfica (club → liga → confederación) y deportiva (jugador → país → posición).
- ***DRILL-DOWN (3):*** desciende en tres pasos independientes (año → trimestre → mes en la Premier, liga → club → jugador en La Liga 2022, confederación → liga → club por valor de mercado).
- ***PIVOT (3):*** simula tabulación cruzada con *FILTER (WHERE ...)* (temporada * liga, club * temporada, tipo de evento * día de la semana).
- ***Funciones ventana (5):*** *SUM() OVER* (acumulado gasto Chelsea), *AVG() OVER ROWS BETWEEN (media móvil 3 meses)*, *LAG()* (evolución valor Mbappé), *NTILE(4)* (cuartiles goleadores), *ROW_NUMBER() PARTITION BY* (mejor goleador por club y liga).
- ***GROUPING SETS / CUBE / ROLLUP avanzado (3):*** cuatro niveles simultáneos con *GROUPING SETS*, las ocho combinaciones dimensionales con *CUBE*(posición, tipo, capitán), y balance compras/ventas con *FULL OUTER JOIN* entre *CTEs*.

6.2.2 Sección 8: Análisis Estratégicos Combinados (5 consultas)

Cada consulta integra múltiples operaciones en un único enunciado analítico:

- ***SLICE + media móvil*** de 6 meses sobre el valor de mercado del Top 5 de La Liga.
- ***DICE + ROLLUP*** comparando productividad goleadora (goles/90 min) entre titulares y suplentes en las Cinco Grandes Ligas.
- ***DRILL-DOWN temporal + LAG*** para variación interanual del gasto en fichajes por liga.
- ***CUBE + ROW_NUMBER*** para el análisis multidimensional de tarjetas por posición y liga.
- ***SLICE + PIVOT + acumulada ventana*** para la matriz de rendimiento mensual del Real Madrid 2023/24.

6.2.3 Sección 9: Consultas Estrella (10 consultas)

Cada consulta incluye en sus comentarios la pregunta de negocio, las operaciones ***OLAP*** empleadas, la visualización recomendada y el *insight* esperado. Los temas cubiertos son:

- Top 25 goleadores y asistentes históricos de las Cinco Grandes Ligas.
- Top 20 fichajes más caros con ratio precio/valor de mercado.
- Ventaja de campo por liga (% victorias locales, empate, visitante).

- Pichichi por temporada y liga con *RANK() OVER (PARTITION BY liga, temporada)*.
- Balance neto de compras y ventas por club con *FULL OUTER JOIN*.
- Distribución de goles por franja de minuto (*CASE WHEN minute ...*).
- Top 10 nacionalidades goleadoras en las Cinco Grandes Ligas.
- Inflación del mercado de fichajes con *PERCENTILE_CONT(0.5)* y *LAG()*.
- Resumen ejecutivo comparativo de las Cinco Grandes Ligas articulado con tres *CTEs* sobre ***fact_games***, ***fact_transfers*** y ***fact_appearances.***

6.2.4 Sección 10 – 12: Extensiones (10 consultas)

- ***Sección 10 (*Auditoría técnica**) (1): cuantifica el uso de registros centinela (*fk = -1*) en las cuatro claves foráneas opcionales mediante *COUNT(*) FILTER (WHERE fk = -1)* y *UNION ALL*.

- ***Sección 11* Análisis individual de jugadores** (5): ranking de apariciones con *ROW_NUMBER* y *NTILE*, rendimiento como capitán vs. no capitán con *PIVOT*, jugadores más nómadas con *STRING_AGG* y *COUNT(DISTINCT club_id) >= 4*, mayores revalorizaciones anuales con *LAG* y *LEAD*, productividad por pie dominante normalizada a 90 minutos.

- ***Sección 12* Rendimiento colectivo y táctico** (4): mejor ataque y defensa histórica de cada liga con *DENSE_RANK() OVER (PARTITION BY liga);* correlación gasto en fichajes / porcentaje de victorias, usando *UNION ALL* para combinar actuaciones como local y visitante y *NTILE(4)* para cuartiles; evolución goleadora por temporada con *LAG*, *RANK* y *AVG() OVER (PARTITION BY liga)* como referencia histórica; top 25 partidos más goleadores con desglose de tarjetas y sustituciones.

Una decisión de diseño transversal a todo el fichero fue usar exclusivamente INNER JOIN, posible gracias a los registros centinela del ETL. Donde estos distorsionarían el análisis, se aplica el filtro explícito *WHERE player_id != -1*, *WHERE club_id != -1* o *WHERE country_of_citizenship != 'N/A'*.

6.3 Técnicas SQL Avanzadas Empleadas

Funciones de Ventana (OVER)

Función	Uso principal
ROW_NUMBER() OVER (PARTITION BY… ORDER BY…)	Rankings por grupo sin subconsultas correlacionadas.
DENSE_RANK() OVER (PARTITION BY liga)	Rankings independientes por competición.
SUM() OVER (ORDER BY temporada)	Acumulados corrientes de gasto
AVG() OVER (ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)	Media móvil suavizada.
AVG() OVER (PARTITION BY liga)	Media histórica de referencia por liga
LAG() / LEAD()	Comparaciones interperiodo (valor de mercado, gasto en fichajes).
NTILE(4)	Cuartiles de producción goleadora y de gasto por club.

***Figura 9**: Tabla con Funciones de Ventana Empleadas.*

Agregaciones multinivel

- *GROUPING SETS*, *CUBE* y *ROLLUP* generan simultáneamente varios niveles de subtotales en una sola consulta, eliminando *UNION ALL* manuales.
- *GROUPING()* acompaña siempre a estas cláusulas para distinguir filas de subtotal de filas de detalle.
- *COALESCE*(comp.name, *'** TODAS LAS LIGAS **'*) etiqueta los *NULLs* de subtotal con literales descriptivos.

CTEs (WITH)

- La consulta 9.10 encadena tres *CTEs* independientes (***goles_liga***, ***fichajes_liga***, ***tarjetas_liga***) antes de combinarlas con *JOIN*.

- La consulta 7.3 separa compras y ventas en dos *CTEs* antes de *FULL OUTER JOIN* para evitar ambigüedades.
- La consulta 12.2 construye ***resultados_por_club*** mediante un *UNION ALL* interno de actuaciones como local y visitante antes de cruzarla con el gasto en fichajes.

Otras técnicas

- ***PIVOT manual:*** *SUM(medida) FILTER (WHERE dimension = 'valor') AS "columna"*, adoptado en las secciones 5, 11.2 y 11.5 al no existir *PIVOT* nativo en *PostgreSQL*.
- ***NULLIF(denominador, 0):*** previene errores de división por cero en todos los ratios por partido y por 90 minutos.
- ***PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY ...):*** mediana del precio de fichaje por año (consulta 9.9), como complemento robusto a la media aritmética frente a valores extremos.

6.4 Problemas Encontrados y Soluciones

Problema 1: Rendimiento inaceptable en rankings por grupo.

- **Descripción**: Las primeras versiones de consultas que extraían el top N por grupo (máximo goleador de cada club en cada liga, Pichichi por temporada) usaban subconsultas correlacionadas en el *WHERE*, con tiempos de ejecución enormes sobre ***fact_appearances*** (1.628.836 registros).
- **Causa**: Una subconsulta correlacionada se re-ejecuta una vez por cada fila del resultado exterior, generando un coste cuadrático inviable a esta escala.
- **Solución**: Sustitución por *ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)* encapsulado en una *CTE*, filtrando después *WHERE rn =1*. La consulta 6.5 documenta explícitamente esta decisión en su comentario: *"sin necesidad de subconsultas correlacionadas"*.

Problema 2: Grupos Fantasma por Registro Centinela.

- **Descripción**: Consultas con *GROUP BY p.country_of_citizenship* o *GROUP BY c.name* producían grupos etiquetados *'N/A'* o *'Desconocido'* que distorsionaban los rankings.
- **Causa**: Los registros centinela(*player_id = -1*, *club_id = -1*), y los atributos de textos rellenos con la constante *DIMENSION_TEXT_DEFAULT = 'N/A'* de null_handler.py participaban en las agregaciones sin filtrado adicional.
- **Solución**: Adición sistemática de *WHERE player_id != -1*, *WHERE club_id != -1* y *WHERE country_of_citizenship != 'N/A'* en todas las consultas analíticas. Patrón visible en consultas 2.1, 9.1, 9.2, 9.8 y en las secciones completas 11 y 12.

Problema 3: Ausencia de *PIVOT* nativo en *PostgreSQL*.

- **Descripción**: La presentación de datos en tabla cruzada (goles por temporada con ligas como columnas, rendimiento por pie dominante) no tiene soporte sintáctico directo en PostgreSQL.
- **Causa**: PostgreSQL no implementa la cláusula *PIVOT* estándar. La alternativa oficial (***crosstab*** de ***tablefunc***) requiere activar una extensión y tiene sintaxis poco legible para columnas dinámicas.
- **Solución**: *SUM(total_goals) FILTER (WHERE competition_id = 'ES1') AS "La Liga"* para cada valor de la dimensión pivotada. Más legible que el equivalente con *CASE WHEN* y sin dependencias externas. Estandarizado en secciones 5, 11.2 y 11.5.

7. 1 DE MARZO AL 2 DE MARZO

Durante los días 1 y 2 de marzo, con una dedicación aproximada de **6 horas repartidas en dos sesiones,** se llevó a cabo la fase de visualización analítica del ***Data Warehouse***. Conforme a la planificación original del proyecto, esta etapa estaba prevista para el período comprendido entre el 24 y el 26 de febrero, con una duración estimada de tres días. Sin embargo, el retraso acumulado durante la fase ***ETL*** (explicado en el apartado 4 (*ETL*) se propagó en cascada sobre el calendario global, desplazando esta etapa hasta los primeros días de marzo. No obstante, el trabajo se completó en dos días, es decir, un día menos de lo inicialmente estimado, lo cual permitió compensar parcialmente el desfase acumulado. En

cuanto a la minería de datos, prevista conjuntamente con la visualización en la planificación inicial, el nivel de detalle, rigor técnico y cuidado estético invertido en las visualizaciones generadas absorbió la totalidad del tiempo disponible en esta fase, razón por la cual dicha línea de trabajo constituye una extensión natural del proyecto de cara a desarrollos futuros.

Respecto a la herramienta empleada, la planificación inicial contemplaba como opción principal la integración con ***Power BI*** o ***Plotly***. Sin embargo, conforme avanzó el desarrollo de las consultas ***OLAP*** y se valoró la flexibilidad necesaria para presentar análisis altamente personalizados, se optó por la alternativa también prevista en la planificación: ***Matplotlib*** en Python, ejecutado dentro de un ***Jupyter Notebook***. Esta elección se alineó además con la arquitectura técnica ya establecida en el proyecto (***Python*** + ***PostgreSQL*** + ***SQLAlchemy***), minimizando la incorporación de nuevas dependencias externas y maximizando el control sobre cada aspecto visual.

7.1 Contexto y Planificación

Con el DW completamente operativo y el corpus de 49 consultas OLAP documentado, el objetivo de esta fase consistió en seleccionar un subconjunto representativo de las consultas más significativas e implementar para cada una de ellas una visualización clara, estética y directamente presentable. El criterio de selección priorizó las consultas de la sección 9 (consultas estrella) y la sección 12 (rendimiento colectivo y táctico), por ser las que responden a preguntas de negocio concretas y producen resultados visualmente más ricos. El resultado fue un total de 10 visualizaciones implementadas en el fichero visualizaciones_olap.ipynb.

7.2 Estructura del Notebook de Visualización

El notebook se organiza en 10 bloques independientes, cada uno compuesto por una celda markdown descriptiva y una celda de código Python. Cada bloque incluye: la sentencia SQL original íntegra (extraída literalmente de consultas_olap.sql), la carga del resultado mediante *run_query*() a través de *SQLAlchemy*, el tratamiento y tipado del *DataFrame* resultante, la generación del gráfico con *Matplotlib* y el guardado del fichero PNG en el directorio graficos/.

#	Consulta	Visualización
1	9.1	Gráfico de barras horizontal - Top 25 goleadores históricos con goles/90 min.
2	9.4	Gráfico de barras agrupadas - Factor de campo por liga (% local/empate/visitante).
3	9.5	Mapa de calor - Pichichi por temporada y liga (RANK por partición).
4	9.6	Gráfico de barras horizontal - Balance neto compras/ventas por club (Big Five).
5	9.7	Gráfico de puntos - Top 10 goleadores por minutos jugados por 90.
6	9.9	Gráfico de líneas con anotaciones - Inflación del mercado de fichajes (mediana anual).
7	9.10	4 subgráficos de barras horizontales - Comparativa Big Five por KPI (goles, asistencia, fichaje y disciplina).
8	12.2	Diagrama de dispersión - Correlación gasto en fichajes vs. porcentaje de victorias por club.
9	12.3	Gráfico de líneas multipanel - Evolución de goles totales por

		temporada y liga, con media histórica y estrella en el máximo
10	12.4	Gráfico de barras horizontal - Top 25 partidos más goleadores con desglose de tarjetas y sustituciones

Figura 10: *Tabla con las Visualizaciones Generadas y su Correspondencia con las Consultas OLAP.*

7.3 Decisiones de Diseño Visual

Se adoptaron un conjunto de decisiones de diseño transversales a todas las visualizaciones para garantizar coherencia estética y legibilidad:

- **Paleta de colores por liga:** se definió un diccionario LIGA_COLORS con un color identificativo para cada una de las Cinco Grandes Ligas (rojo para La Liga, azul para la Premier League, verde para la Serie A, naranja para la Bundesliga y púrpura para la Ligue 1), aplicado de forma consistente en todos los gráficos que desglosan por competición.
- **Guardado automático:** la función auxiliar save_fig() centraliza el guardado de cada figura en graficos/ con resolución de 150 DPI, permitiendo su uso directo en documentos sin postprocesado adicional.
- **Anotaciones de valor:** en barras, dispersión y líneas se añadieron etiquetas con el valor numérico exacto sobre cada elemento, eliminando la necesidad de que el lector interpole visualmente.
- **Leyendas externas al área de datos:** en los gráficos con mayor densidad visual (9.6, 9.10), las leyendas se posicionaron fuera del área de trazado mediante bbox_to_anchor, evitando solapamientos con los datos.
- **adjustText:** en la visualización 12.2 (dispersión de 30 clubes), se empleó la librería adjustText para reposicionar automáticamente las etiquetas de texto y eliminar colisiones, garantizando que los 30 clubes fueran legibles sin solapamiento.

7.4 Problemas Encontrados y Soluciones

Problema 1: Solapamiento de etiquetas en el diagrama de dispersión (12.2).

- **Descripción:** Con 30 clubes representados simultáneamente, las etiquetas de texto se solapaban entre sí, haciendo el gráfico ilegible.
- **Solución:** Instalación e integración de la librería ***adjustText***, que aplica un algoritmo iterativo de repulsión entre textos, ajustando automáticamente la posición de cada etiqueta hasta eliminar colisiones.

Problema 2: Posicionamiento incorrecto de la leyenda en gráficos de barras horizontales (9.6).

- **Descripción:** En el gráfico de balance neto, la leyenda se superponía al área de barras independientemente de los valores de *loc* estándar, al ser el panel muy largo verticalmente.
- **Solución:** Uso de *bbox_to_anchor=(0, 1.02)*, *loc='lower left'* combinado con *fig.subplots_adjust(top=0.84)*, anclando la leyenda completamente fuera del área de datos, por encima de los nombres de los equipos.

Problema 3: Estrella mal posicionada en la visualización de temporada goleadora (12.3).

- **Descripción:** El marcador de mejor temporada se situaba visualmente en un punto bajo de la curva, al basarse en el ranking por ***promedio_goles*** de la consulta SQL mientras el eje Y mostraba ***goles_totales***.
- **Causa:** La consulta SQL *rankea* por promedio de goles por partido (métrica relativa), pero la visualización más informativa era el total absoluto de goles, que no tiene por qué coincidir con el máximo del promedio (una temporada con muchos partidos puede tener mayor total, aunque su promedio sea inferior).
- **Solución:** La estrella se posicionó mediante *sub.nlargest(1, 'goles_totales'),* desvinculándola del ranking SQL y anclándola al máximo real del eje Y representado, garantizando coherencia visual entre el marcador y la curva.

8. BIBLIOGRAFÍA

[1] Kimball, R., & Caserta, J. (2004). The Data Warehouse ETL Toolkit: Practical techniques for extracting, cleaning, conforming, and delivering data. Wiley.

[2] Kimball, Ralph; Margy Ross (2013). The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling (3rd ed.). Wiley.

[3] Kimball Group. (s. f.). Dimensional Modeling Techniques. Último Acceso marzo de 2026.